

Executive Summary

Backstage provides built-in actions and official plugins for Git operations, but advanced conflict handling and CLI integration (for tools like the OpenChoreo `occ` CLI) typically require custom logic. Official action modules (e.g. `@backstage/plugin-scaffolder-backend-module-github`) include actions like **publish:github:pull-request**, which can create a branch, commit workspace files, and open a PR in one step [17†L17-L25] [85†L75-L84]. These actions accept inputs (branch name, commit message, target path, etc.) and return the new PR number/URL [17†L17-L25] [17†L33-L36]. Roadie's "GitHub Pull Requests" UI plugin can display PR lists and stats (time to merge, size, etc.) [26†L19-L23] but does *not* create or update PRs.

To invoke an external CLI like `occ` from a template, the recommended approach is a **custom backend action**. In its handler, you can spawn the CLI using Node's `child_process`, writing output files into `ctx.workspacePath` (e.g. using `fs-extra` [61†L169-L177]) and streaming stdout/stderr via `ctx.logger`. Credentials (e.g. Git tokens) should be passed via Backstage's secrets or SCM auth (not hard-coded). Before opening a PR, your action should detect conflicts: for example, check for existing open PRs in the repo (using GitHub/GitLab APIs) and reuse/update them if appropriate (e.g. use `update: true` [17†L33-L36] [69†L179-L181]), or create a uniquely named branch otherwise. We recommend pseudocode that lists open PRs, filters by branch or target path, and either updates the PR (pushing new commits) or creates a new branch+PR.

Key findings:

- **Built-in Actions** – Spotify/Backstage scaffolder has "publish" actions for GitHub, GitLab, Bitbucket, etc., including a dedicated `publish:github:pull-request` action [17†L17-L25] [85†L75-L84]. These handle branch creation, commit, and PR creation.
- **Roadie PR Plugin** – This UI plugin shows PRs and metrics [26†L19-L23] but offers no PR-creation automation. It requires GitHub auth and is archival (read-only).
- **External CLI Integration** – No out-of-box `run:command` action exists; instead write a custom action that calls `occ`. Handle paths via `ctx.workspacePath` [61†L169-L177], stream logs with `ctx.logger`, and enforce security by scoping tokens.
- **Conflict Detection** – Check existing branches/PRs via the SCM API. The GitHub action supports an `update: true` flag to update an existing PR [17†L33-L36] [69†L179-L181]. Pseudocode will iterate open PRs, compare branches/paths, and decide to update or open new.
- **Existing Examples** – Roadie's docs show using `update: true` [69†L179-L181]. No official plugin auto-resolves conflicts; typical solutions use Octokit calls in custom actions to list PRs/branches and act accordingly.
- **Architecture** – We propose a Scaffolder template with steps to validate inputs, run `occ`, write YAML to `workspace`, then a `publish:github:pull-request` step. The custom `occ` action (TypeScript) handles CLI invocation and outputs the commit/branch info. Permissions must include Git write scopes (`repo`, etc.), and secrets (like `GITHUB_TOKEN`) should use Backstage's secure store (see token inputs [20†L23-L26]). Include retry/backoff logic on transient failures.

- **Testing** – Unit-test the custom action handler with `createMockActionContext` [89†L74-L82] . Integration tests can run with a real `GITHUB_TOKEN` against a throwaway repo. Dry-run mode can simulate actions without side-effects [91†L65-L73] . CI should cover unit, integration, and a full e2e run.
- **Risks/Alternatives** – Using built-in actions is easiest but may lack advanced merge conflict logic. A custom action offers full control but requires maintenance. Roadie's UI plugin enhances visibility but does not automate PR creation. Alternatives include writing a dedicated GitHub App, using ArgoCD's PR generator feature, or external tools like Kargo for promotion workflows. We compare approaches below.

A recommended MVP approach is to **use the built-in Scaffolder action (`publish:github:pull-request`)** for PR creation, combining it with a custom Occ-generation action for file output. This minimizes new code and leverages Backstage's built-in auth/logging, while allowing the custom conflict-resolution logic in the template or action.

1. Backstage Built-in Scaffolder Actions and Official Plugins

Backstage provides several **official scaffolder action modules** for Git providers [85†L75-L84] . For GitHub, the `@backstage/plugin-scaffolder-backend-module-github` module exposes actions such as:

- `createPublishGithubAction` (**`publish:github`**): Initializes a new repo and pushes workspace contents; returns commit info and URL.
- `createPublishGithubPullRequestAction` (**`publish:github:pull-request`**): Creates a branch in an existing repo, commits workspace contents (to a given path), and opens a PR [17†L17-L25] . Inputs include:
 - `repoUrl` (e.g. `github.com?repo=repo&owner=org`)
 - `branchName` (name of the new branch)
 - `commitMessage`, `title`, `description` for the PR
 - `sourcePath` (workspace subfolder to publish) and `targetPath` (path in the repo)
- Optional: `reviewers`, `teamReviewers`, `draft`, `filesToDelete`, and an `update` flag [17†L17-L25] [17†L33-L36] . It returns `{ pullRequestNumber, remoteUrl, targetBranchName }` [17†L17-L25] . For example, the docs show:

Inputs: `branchName`, `commitMessage`, `title`, `description`, `sourcePath`, `targetBranchName`, `targetPath`, `reviewers`, `update`, etc.

Outputs: `pullRequestNumber`, `remoteUrl`, `targetBranchName` [17†L17-L25] [17†L33-L36] .

- **Other modules:** Similar actions exist for GitLab (`createPublishGitlabPullRequestAction` in `plugin-scaffolder-backend-module-gitlab`), Bitbucket Cloud/Server, Azure DevOps, Gitea, etc [85†L75-L84] . They offer equivalent “push and PR” actions (often called Merge Request for GitLab).

The built-in **`publish:github:pull-request`** (Roadie usage page) allows publishing to subdirectories in a monorepo as well (via the `targetPath` input) [69†L229-L238] . In summary, **branch creation, commit,**

and PR are directly supported by these official plugins, requiring only configuration of inputs in your template [17†L17-L25] [85†L75-L84] .

2. Roadie GitHub Pull Requests Plugin (Capabilities & Limitations)

Roadie provides a **GitHub Pull Requests plugin** for Backstage which is *front-end only*. Its capabilities include:

- **Listing and searching PRs:** On an entity page (e.g. service component), it shows open PRs using GitHub's query syntax [26†L12-L20] .
- **Metrics and charts:** It displays stats like average time to merge, merge vs close ratio, lines/files changed, and a "time from first commit to PR creation" metric [26†L19-L23] .
- **Overview and group cards:** It offers an "Overview" card and a team/group PR board that aggregates PRs across repos.
- **GitHub Enterprise support:** Works with GitHub Enterprise by pointing to a custom host.

However, **it does NOT create or update PRs**. It is a read-only view of existing GitHub data. It requires the GitHub auth provider to be configured (sign-in) [26†L19-L23] . The plugin is primarily for *visibility*, not for workflow automation. In short, Roadie's PR plugin makes reviewing and monitoring PRs easier, but teams still need to use the scaffolder or other tools to *open* PRs.

Furthermore, the plugin has been moved to a Roadie-maintained monorepo (and archived) [27†L128-L137] , so there's no active development by the community beyond maintenance. Its limitations for our use-case are clear: it won't help run `occ` or generate the PR — it only displays PRs after the fact. We cite Roadie's documentation on its listing and stats features [26†L19-L23] .

3. Invoking External CLIs (occ) from Scaffolder

To run an external CLI like `occ` from within a Backstage template, the typical pattern is to use a **custom backend action**. (Currently there is no simple `run:command` built-in action that works cross-platform out-of-the-box.) Two approaches:

- **Built-in (experimental) execute:** Backstage scaffolder has introduced an action to run shell commands (e.g. `execute:shell`), but its availability and cross-platform behavior can be limited. Using it requires careful path/command setup. For full control, we recommend writing a custom action.
- **Custom action:** Use `createTemplateAction` in a backend module. In the action's `handler(ctx)` you can use Node's `child_process` to spawn the CLI. For example:

```
import { createTemplateAction } from '@backstage/plugin-scaffolder-backend';
import { spawn } from 'child_process';
export const createOccAction = () =>
  createTemplateAction({
    id: 'mycompany:occ:generate',
```

```

description: 'Run OpenChoreo CLI to generate manifests',
schema: { input: { /* define input schema */ } },
async handler(ctx) {
  const occArgs = [ /* from ctx.input values */ ];
  const proc = spawn('occ', occArgs, { cwd: ctx.workspacePath });
  proc.stdout.on('data', data => ctx.logger.info(data.toString()));
  proc.stderr.on('data', data => ctx.logger.error(data.toString()));
  const exitCode = await new Promise<number>(resolve =>
    proc.on('close', resolve),
  );
  if (exitCode !== 0) {
    throw new Error(`occ command failed with exit code ${exitCode}`);
  }
  // Occ has produced files in the workspace; continue
},
});

```

In this handler:

- Use `ctx.workspacePath` as the working directory (use `resolveSafeChildPath(ctx.workspacePath, ...)` if writing new files) [\[61†L169-L177\]](#) .
- Stream logs to `ctx.logger` so they appear in the Backstage UI logs (debug/error/info as appropriate).
- Ensure **security/credentials**: do not embed any sensitive credentials in the action. If `occ` or downstream processes need secrets, pass them via the scaffold template's `values` (marked as secret) or the system's secret store. For Git operations later, use `${{ secrets.githubToken }}` or similar (see inputs below).
- **Cross-platform**: If Backstage backend runs on Windows, ensure your command invocation is compatible (for instance, use `spawn('occ.cmd', ...)` or `shell: true`). Using a JavaScript CLI (if `occ` were Node-based) is easier; for native CLIs, test on target OS or use containerized build agents.
- **Error handling**: The action should catch and throw errors if the CLI fails (non-zero exit). Backstage will mark the step as failed and prevent PR creation. Optionally, implement retry/backoff for transient errors (e.g. if calling a remote service).

We also mention **dry-run support**: You can set `supportsDryRun: true` and check `ctx.isDryRun` to simulate the action without side-effects [\[91†L74-L82\]](#) [\[91†L140-L148\]](#) .

In summary, invoking `occ` will be done in the template workflow via a custom action step that handles file generation securely. We cite Backstage's custom action guide to show writing to the workspace [\[61†L169-L177\]](#) .

4. Conflict Detection Strategies

Before pushing changes and creating a PR, the backend should check for conflicts. Strategies include:

- **Unique branch naming:** Generate a branch name that is unlikely to collide, e.g. include a timestamp, random suffix, user ID, or ticket number. For example: `create-workload-${{ values.serviceId }}-${{ env.USER }}-${{ parameters.timestamp }}`. This “optimistic” approach reduces branch-name collisions but still may conflict if rerun quickly. Always check if the branch already exists.
- **Check existing branches/PRs:** Use the Git provider API (e.g. GitHub’s REST/GraphQL) to list open PRs on the target repo/target branch. Filter by head branch name or by `targetPath`. For example, using Octokit:

```
const { data: prs } = await octokit.rest.pulls.list({
  owner, repo, state: 'open', base: targetBranch,
});
const existing = prs.find(pr => pr.head.ref === branchName);
```

If `existing` PR is found, consider *updating* it rather than making a new one. (The `update: true` input in the GitHub action automates this.) Also check if any open PR touches the same files or `targetPath` – if so, you may need to merge or coordinate.

- **Optimistic vs Safe naming:** If update mode is desired (e.g. for automation that regenerates manifests periodically), use a fixed `branchName` (like “chore/whatevery”) and set `update: true` `【17†L33-L36】` `【69†L179-L181】`. This causes the action to reuse that branch and PR. Otherwise, generate a unique name and leave `update: false`.
- **Mergeability checks:** After creating/updating a PR, you can poll the PR’s `mergeable` state via the API to see if there are conflicts. GitHub’s API returns `mergeable: 'CONFLICTING'` if a PR cannot merge cleanly. If so, your action could optionally update the branch with the latest target branch or abort and report an error.
- **Pre-merge validation:** Locally attempt a `git merge` or `git rebase` (via the API or by cloning) to simulate merge conflicts before making the PR. If conflicts arise, either rename the branch or signal failure. This is advanced and may be skipped for MVP.

A **recommended algorithm (pseudocode)** might be:

```
function prepareBranchAndPR(repo, baseBranch, changes) {
  let branchName = generateUniqueBranchName(baseBranch);

  // Check for existing open PR for this branchName
  existingPR = findOpenPullRequest(repo, branchName, baseBranch);
```

```

if (existingPR) {
  // Update existing PR
  console.log(`Updating existing PR #${existingPR.number}`);
  updatePullRequest(existingPR.number, changes);
  return existingPR.url;
}

// Otherwise, ensure branch name is indeed unique
if (branchExists(repo, branchName)) {
  branchName = generateAnotherBranchName();
}

// Create new branch from base
createBranch(repo, baseBranch, branchName);
commitChangesToBranch(repo, branchName, changes);

// Create a new PR
pr = createPullRequest({
  owner, repo, head: branchName, base: baseBranch,
  title, body, draft: false,
});
return pr.url;
}

```

Key steps: check for an existing PR on the chosen branch (or matching changes), use `update: true` to push new changes if it exists [69†L179-L181], else create branch and PR. We would encode this logic either by setting inputs (`branchName`, `update: true`) or explicitly via API calls in a custom action.

5. Existing Plugins / Community Examples for Conflict/PR Handling

No mainstream Backstage plugin currently auto-resolves merge conflicts or updates PRs beyond the built-in flags. However:

- The official GitHub scaffolder action itself supports updating PRs (the `update: true` flag) [17†L33-L36] [69†L179-L181]. Roadie's examples demonstrate using this for recurring updates.
- **Custom actions with Octokit:** In the Backstage codebase and community examples, teams have written custom actions that use the GitHub (or GitLab) API. For instance, a custom TypeScript action could use `octokit.rest.pulls.list` to find an existing PR, and then call `octokit.rest.pulls.update` or simply push new commits to the branch (which auto-updates the PR). There isn't an official reference guide for this, but it follows standard GitHub API usage.

Here's a minimal illustrative snippet (conceptual):

```

// In a custom action handler:
const [owner, repo] = ctx.input.repoUrl.match(/owner=([^&]*)&?/?).[1];
// List open PRs for the repo

```

```

const { data: prs } = await octokit.rest.pulls.list({ owner, repo, state:
  'open' });
const pr = prs.find(p => p.head.ref === branchName);
if (pr) {
  // Existing PR found, push new commit to branch (ctx already did commit step)
  ctx.output('pullRequestNumber', pr.number);
  ctx.output('remoteUrl', pr.html_url);
} else {
  // No existing PR, use built-in action or API to create one
  const created = await octokit.rest.pulls.create({
    owner, repo,
    head: branchName, base: baseBranch,
    title: prTitle, body: prDesc, draft: false,
  });
  ctx.output('pullRequestNumber', created.data.number);
  ctx.output('remoteUrl', created.data.html_url);
}

```

In the absence of a specific ready-made plugin for conflict resolution, these patterns are implemented ad-hoc. Roadie's plugin for listing PRs [26†L19-L23] or the backend GitHub action with `update: true` [17†L33-L36] are the closest official tools. We find no published Backstage plugin solely for conflict detection/resolution. The main community examples (like Roadie's usage docs [69†L179-L181]) simply show using `update: true` to reuse PRs.

6. Recommended Template and Action Design

A **Scaffolder template** (YAML) should orchestrate the steps:

```

apiVersion: backstage.io/v1alpha1
kind: Template
metadata:
  name: openchoreo-workload
  title: Create OpenChoreo Workload
spec:
  owner: team@example.com
  parameters:
    - title: Workload Name
      name: workloadName
      type: string
      required: true
    - title: GitHub Repo Owner
      name: repoOwner
      type: string
      required: true
    - title: GitHub Repo Name
      name: repoName

```

```

    type: string
    required: true

steps:
  - id: validate-inputs
    action: builtin:noop # (Could be a custom input validation action if
needed)
    input: {}

  - id: generate-openchoreo
    action: myorg:occ:generate
    input:
      # pass parameters or values needed by occ
      workloadName: ${ parameters.workloadName }

  - id: publish-to-github
    action: publish:github:pull-request
    input:
      repoUrl: github.com?repo=${ parameters.repoName }&owner=${
parameters.repoOwner }
      branchName: "openchoreo-${ parameters.workloadName }-${
parameters.timestamp }"
      targetBranchName: main
      title: "Automated OpenChoreo workload for ${
parameters.workloadName }"
      description: "Generated by Backstage scaffolder"
      sourcePath: ./ # the workspace (occ output) root
      targetPath: "${ parameters.workloadName }/" # or wherever in repo to
place files
      commitMessage: "chore: add workload ${ parameters.workloadName }"
      # Set to true if wanting to update an existing PR for the same branch
      update: false
      # Pass GitHub token securely (see below)
      token: ${ secrets.GITHUB_TOKEN }

```

In this design:

- The first custom action `myorg:occ:generate` runs `occ` and writes all generated YAML into `ctx.workspacePath` (the default is `./`).
- The second built-in action `publish:github:pull-request` commits that workspace content into a new branch and opens a PR. We supply `repoUrl`, `branchName`, commit message, etc.
- We use a unique branch name (including workloadName and a timestamp) to avoid collisions.
- We set `update: false` here, but if this template is re-run with the *same* branchName, one could use `update: true` to push to the same PR [\[69†L179-L181\]](#) .

Backend action code (TypeScript skeleton):

```

import { createTemplateAction } from '@backstage/plugin-scaffolder-backend';
import { resolveSafeChildPath } from '@backstage/backend-plugin-api';
import fs from 'fs-extra';
import { spawn } from 'child_process';

export const createOccGenerateAction = () =>
  createTemplateAction({
    id: 'myorg:occ:generate',
    description: 'Run OpenChoreo CLI to produce YAML',
    schema: {
      input: {
        type: 'object',
        required: ['workloadName'],
        properties: {
          workloadName: { type: 'string', title: 'Workload Name' },
        },
      },
    },
    async handler(ctx) {
      const workload = ctx.input.workloadName;
      // Assume occ CLI is available in PATH or provide full path
      const occArgs = ['generate', 'workload', workload];
      const proc = spawn('occ', occArgs, { cwd: ctx.workspacePath });

      // Stream stdout/stderr to scaffolder log
      proc.stdout.on('data', data => ctx.logger.info(data.toString()));
      proc.stderr.on('data', data => ctx.logger.error(data.toString()));

      const exitCode: number = await new Promise(resolve =>
        proc.on('close', resolve),
      );
      if (exitCode !== 0) {
        throw new Error(`occ CLI failed (exit ${exitCode})`);
      }

      // Optionally, post-process files in workspace
      // e.g. merge multiple YAMLS, etc.
    },
  });

```

Permissions & Secrets: To create a PR on GitHub, the action needs a token with `repo` scope (for private repos) or `public_repo` (for public), plus `workflow` scope if modifying GitHub Actions workflows. In Backstage, we supply `{{ secrets.GITHUB_TOKEN }}` [201L23-L26] as an input, which should come from the backend's credential store or CI environment. Similarly, for GitLab use the `api` scope token. For security, the template should treat tokens as secrets. Do **not** hard-code tokens.

Retry/Backoff: We recommend retrying transient API calls (e.g. in case of rate-limit or network issues), using exponential backoff. For simplicity, the built-in actions automatically retry some GitHub errors; in a custom action you could wrap Octokit calls with retry logic.

7. Testing Plan and CI

- **Unit tests:** Test the custom action handler logic in isolation. Use `createMockActionContext` from `@backstage/plugin-scaffolder-node-test-utils` to mock `ctx` [89†L74-L82]. For example, supply a fake workspace with sample files or inputs, run `handler()`, and assert that the action calls `ctx.output` or writes the correct files. Mock external calls (spawn/Octokit) to simulate different outcomes.
- **Integration tests (local):** Run Backstage with a real `GITHUB_TOKEN` pointing to a test repository. Execute the template (via Backstage UI or CLI) and verify that a branch and PR are created as expected. Also test the “dry-run” mode by setting `supportsDryRun: true` and `ctx.isDryRun = true` to ensure no actual Git changes happen [91†L140-L148].
- **CI pipeline:** Include a workflow that runs `yarn test` (unit), then perhaps a Dockerized Backstage instance executing the template on a sandbox repo. Use GitHub Actions or similar with `secrets.GITHUB_TOKEN`. After creating a PR in the test repo, the CI can merge it back or delete branches. Ensure cleanup (delete test branches) to keep the repo tidy.
- **E2E tests:** If feasible, use a real GitHub/GitLab test project. Trigger the entire workflow as a user would (for example, via the scaffolder REST API or by clicking in the UI), then verify the PR outcome (title, branch, files changed). This catches any missing bits in the full system.

Backstage’s “Writing Tests for Actions” guide gives patterns for mocking contexts and inputs [89†L74-L82]. Utilize that to test error cases (e.g. `occ` failure) and success flows. Also test the PR-creation step: you can mock GitHub API responses or use a real sandbox repo to observe behavior.

8. Risks, Limitations, and Alternatives

Risks/Limitations: The main risk is merge conflicts and race conditions. Backstage actions don’t auto-resolve conflicts – if two runs target the same files, a PR may fail to merge. The built-in actions do not do automatic rebases. There’s also the risk of spamming PRs if unique branch names aren’t managed well. Use `update: true` judiciously to avoid duplicate PRs. Security-wise, exposing tokens is a risk; we mitigate it by using Backstage’s secret management and limited scopes [20†L23-L26].

Alternatives:

- **Roadie Plugin:** Only for viewing, not PR creation – it *complements* scaffolder but does not replace its need for automation.
- **GitHub App:** Developing a GitHub App to perform these operations (with fine-grained permissions) is possible. Apps can be more secure (using installation tokens) and can respond to webhooks.

However, writing and maintaining a custom GitHub App is a lot of work compared to using Backstage's scaffolder.

- **ArgoCD PR Generator:** Argo CD has a "PR Generator" feature that can automate repository updates when new image versions appear. If your workflow is image-based, this could be used instead of scaffolder. However, it operates independently of Backstage and typically reacts to cluster changes, not direct developer input.
- **External Promotion Tools (Kargo):** Kargo is an external CI/CD promotion tool (by Fairwinds). It manages application promotion across environments with approvals. It could take the place of manual PRs but requires adopting a whole new tool. It integrates with GitOps but is outside Backstage.

Comparison Table:

Approach	Ease of Implementation	Security	Conflict Handling	Observability	Maintainability	MVP Recommendation
Backstage Built-in (scaffolder) Action	<i>Moderate.</i> Just configure actions in template. No new code for PR.	Uses Backstage auth/credentials. Scopes are limited to tokens you supply [20+L23-L26] .	<i>Basic.</i> Supports <code>update: true</code> for reuse, but no auto-merge on conflicts.	Logs visible in Backstage UI; built-in actions report status.	Low (Backstage maintained). One repo for all logic.	Recommended. Leverage existing action module.
Roadie GitHub PR Plugin (UI)	<i>Low.</i> Easy to install and use. (No coding.)	Uses GitHub auth via Backstage sign-in.	<i>None.</i> It only displays PRs; does not handle creation conflicts.	Good for monitoring PR metrics in UI [26+L19-L23] .	Roadie-maintained. Data access limited to GitHub content.	Useful as a dashboard, not for automation.
Custom Scaffolder Action	<i>High.</i> Need to write and test TypeScript handler.	Depends on implementation; must secure tokens and inputs.	<i>Flexible.</i> You implement any logic (e.g. check, rebase) in code.	Logs can be integrated into Backstage UI.	Higher (you maintain code). Powerful if done right.	Good long-term, but more work; maybe post-MVP.
GitHub App	<i>Very High.</i> Requires separate setup (Auth, webhooks, etc.).	Uses app tokens, fine-grained perms.	Can update PRs similar to above.	Outside Backstage; would need its own UI or logs.	High (two codebases: Backstage & App).	Not recommended for MVP (complex).

Approach	Ease of Implementation	Security	Conflict Handling	Observability	Maintainability	MVP Recommendation
ArgoCD PR Generator	<i>High.</i> Requires using ArgoCD tooling.	Runs with service account permissions.	Designed for automated updates, can merge.	Outside Backstage; ArgoCD UI.	High (must operate ArgoCD).	Unlikely, unless already using ArgoCD heavily.
Kargo (External Tool)	<i>Very High.</i> Entirely separate system to adopt.	Security separate, uses Git credentials.	Conflict resolution depends on tool's features.	Not integrated into Backstage.	High (new system training/maintenance).	Only if already committed to GitOps pipeline with Kargo.

Summary Recommendation: For an MVP, use the **built-in Scaffolder action** (`publish:github:pull-request`) plus a small custom action for `occ`. It has moderate effort (template config + a bit of code) and built-in observability/logging. It handles PR creation and updating (`update: true`) without needing new infrastructure. The Roadie UI plugin can be added later for visibility but is not an alternative for this use-case. GitHub Apps or GitOps tools like Kargo offer more automation but at much greater complexity, so we recommend delaying those until after prototyping with Backstage scaffolder.

Code Snippets:

- *Scaffolder Template (YAML):* See example in Section 6 above.
- *Custom Action (TypeScript):* See `createOccGenerateAction` in Section 6.
- *Conflict-resolution pseudocode:* As outlined in Section 4 (find or create PR logic above).

Sources: Official Backstage docs on actions [\[17†L17-L25\]](#) [\[85†L75-L84\]](#), Roadie plugin guides [\[26†L19-L23\]](#) [\[69†L179-L181\]](#), and Backstage custom-action testing documentation [\[61†L169-L177\]](#) [\[89†L74-L82\]](#) inform these recommendations.